

University of Westminster

Client Server Architecture

5COSC022W

Student Name – Mohammad Sheriffdeen Zaki

UOW Number – w2120655

IIT Number – 20231638

Table of Contents

Part 1: Service Architecture & Setup.....	2
Part 2: Room Management	3
Part 3: Sensor Operations & Linking	4
Part 4: Deep Nesting with SubResources	5
Part 5: Advanced Error Handling & Observability	6

Part 1: Service Architecture & Setup

Explain the default lifecycle of a JAXRS Resource class.

In the JAXRS framework used for this API (Jersey), resource classes like `SensorRoom` and `SensorResource` follow a `RequestScoped` lifecycle by default. This means the runtime instantiates a fresh object for every incoming HTTP request, invokes the matching method, and discards the object for garbage collection once the response is sent.

Architectural Impact: This stateless approach is fundamental to RESTful scalability, as it allows the server to handle thousands of concurrent requests without the overhead of maintaining session state.

Synchronization Strategy: Because resources are recreated, data meant to persist such as our room and sensor lists must be managed by a Singleton Service (`InventoryService`). To prevent data loss or race conditions during high-frequency telemetry bursts, I have utilized thread-safe `ConcurrentHashMap` structures and explicit synchronized blocks on critical update paths (e.g., adding a sensor reading to a sensor's history).

Why is the provision of "Hypermedia" (HATEOAS) considered a hallmark of advanced RESTful design?

HATEOAS (Hypermedia as the Engine of Application State) is the final stage of REST maturity because it makes an API self-documenting at the machine level.

Benefit to Client Developers: Unlike static documentation, which can become outdated, HATEOAS provides dynamic links (e.g., the `_links` map in my `DiscoveryResource`) that guide the client on available actions based on the current state of the resource.

Decoupling: It allows the URI structure to evolve without breaking clients; as long as the client "follows the links" provided in the response rather than hardcoding URIs, the server can rename or move resources with zero impact on the client application.

Part 2: Room Management

When returning a list of rooms, what are the implications of returning only IDs versus returning the full room objects?

This choice represents a tradeoff between Network Efficiency and Client Complexity.

IDOnly: Minimizes initial payload size, which is critical for lowpower IoT devices or highlatency mobile networks. However, it creates a "Chatty API" problem where the client must perform "N+1" requests to retrieve details for every room in the list, increasing total latency.

Full Object: My implementation returns full objects (including sensor IDs and HATEOAS links) to reduce roundtrips. This empowers the client to display a comprehensive dashboard immediately, though it does increase the initial bandwidth consumption.

Is the DELETE operation idempotent in your implementation?

Yes, the DELETE operation is idempotent.

Justification: Idempotency requires that the state of the server remains the same regardless of how many times the operation is called after the first success. In my SensorRoom class, the first call to DELETE /rooms/LIB301 removes the room (State: Deleted). Subsequent calls for the same ID will find that the room no longer exists; while the HTTP status code might change (from 204 to 404), the serverside state remains "Deleted". No additional side effects occur, ensuring the system remains stable even if a client retries a request due to a network timeout

Part 3: Sensor Operations & Linking

Explain the technical consequences if a client attempts to send data in a format other than JSON (e.g., XML).

Because the POST method in SensorResource is strictly annotated with `@Consumes(MediaType.APPLICATION_JSON)`, JAXRS will immediately reject nonJSON payloads.

JAXRS Handling: The runtime performs content negotiation; if the client's `ContentType` header (e.g., `application/xml`) does not match the allowed media type, the server returns an HTTP 415 Unsupported Media Type status.

Safety Benefit: This acts as a primary security layer, preventing the business logic from processing malformed data that could lead to deserialization vulnerabilities or server crashes.

Contrast the query parameter approach with path parameters for filtering.

Path Parameters: These identify a unique resource or a strict hierarchy (e.g., `/rooms/LIB301`). Using them for filtering (e.g., `/sensors/type/CO2`) creates brittle URIs that are difficult to extend if multiple filters (like status or value range) are needed.

Query Parameters: Using `@QueryParam("type")` in SensorResource is the standard RESTful pattern for searching and filtering collections. It allows for flexible, multicriteria queries (e.g., `?type=CO2&status=ACTIVE`) without cluttering the URL path or complicating the routing logic.

Part 4: Deep Nesting with SubResources

Discuss the architectural benefits of the SubResource Locator pattern.

The SubResource Locator pattern (demonstrated by `getReadingResource` in `SensorResource`) is essential for managing complexity as an API grows.

Avoiding "God Classes": Instead of one massive controller class handling every possible path, this pattern allows us to delegate readingspecific logic to a specialized `SensorReadingResource` class.

Context Propagation: It allows the parent context (the `sensorId`) to be passed directly to the child resource, ensuring that reading operations are always scoped to the correct parent sensor, which enhances data integrity.

Part 5: Advanced Error Handling & Observability

Why is HTTP 422 more semantically accurate than 404 for missing references in valid payloads?

A 404 Not Found status typically indicates that the requested URL endpoint does not exist.

Precision: In my API, when a client POSTs a sensor with a nonexistent roomId, the endpoint is valid, but the data is semantically broken. Using HTTP 422 Unprocessable Entity (via `LinkedResourceNotFoundMapper`) tells the developer exactly what happened: "The request is syntactically correct, but it cannot be processed due to a broken logical dependency". This significantly reduces debugging time compared to a generic 404.

Explain the cybersecurity risks of exposing raw Java stack traces.

Raw stack traces are a form of information disclosure that an attacker can exploit.

Gathered Information: A trace reveals the specific versions of libraries (Jersey, Tomcat), absolute file paths on the server's disk, and the exact lines of code where logic fails. This "roadmap" allows attackers to target known CVEs (Common Vulnerabilities and Exposures) or craft precise exploits. My `GlobalExceptionHandler` mitigates this by intercepting all `Throwable` errors and returning a sanitized JSON response

Why use JAXRS filters for logging rather than manual Logger statements?

Using a centralized `LoggingFilter` (implementing `ContainerRequestFilter` and `ContainerResponseFilter`) provides several architectural advantages:

Separation of Concerns: It keeps the business logic in the resources clean and focused on campus management rather than boilerplate logging.

Dry Principle: Updates to the logging format (e.g., adding a timestamp) only need to be made in one file rather than dozens.

Guaranteed Coverage: Unlike manual statements, a filter acts as a "gatekeeper" that is guaranteed to log every transaction, including those that fail before reaching the resource method, providing complete observability.